SESSION 05

K-Nearest Neighbors (KNN)

Classification · Similarity · Decision Boundaries

AI MASTER CLASS

What We'll Cover Today

01 **What is KNN?**

The core intuition — learning by similarity

02 **How KNN Works**

Step-by-step algorithm walkthrough

03 **Distance Metrics**

Euclidean, Manhattan & more

04 **Choosing K**

The most important hyperparameter

05 **KNN in Python**

scikit-learn implementation

06 **Pros & Cons**

When to use KNN — and when NOT to

Quick Recap: Supervised Learning

Training Data

Labeled examples — we know the correct answers
(e.g. spam / not spam emails)

→

Model Training

The algorithm learns patterns from the labeled data to map inputs to outputs

→

Prediction

Given NEW, unseen data — the model predicts the correct label or value

Two Types of Supervised Learning:

REGRESSION Predicts a number (e.g. house price: $350,000)

CLASSIFICATION Predicts a category (email: Spam / Not Spam) KNN does this!

What is KNN? The Core Intuition

Real-Life Analogy: *"Tell me who your friends are, and I'll tell you who you are."*

KNN classifies a new data point by looking at the K most similar (nearest) points in the training data, then taking a majority vote of their labels.

1 → 2 → 3 → 4

1 New point arrives

We receive a data point with unknown label

2 Measure distances

Calculate distance to ALL training points

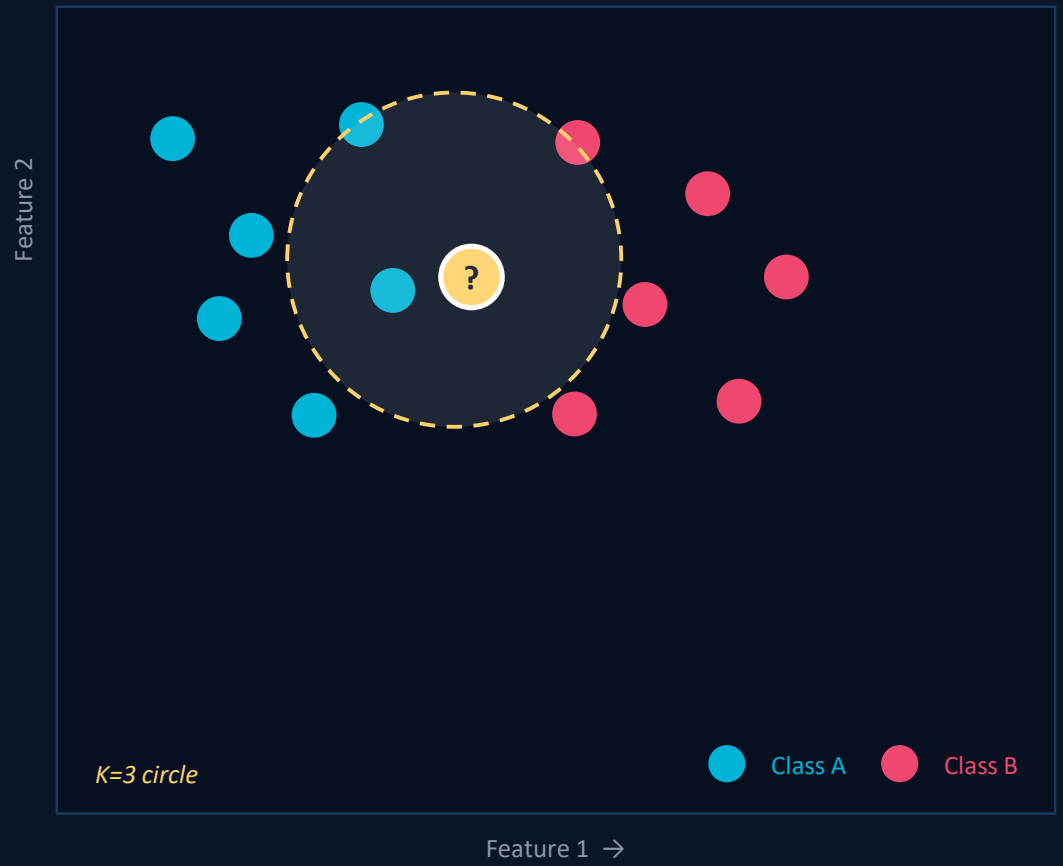
3 Find K neighbors

Select the K closest points (e.g. K=5)

4 Vote!

The majority class among K neighbors wins

KNN in Action — Visual Example (K = 3)



Step by Step:

Yellow point = NEW Unknown data point we want to classify.

Draw K=3 circle Capture exactly 3 nearest neighbors.

Count: 2 blue (Class A) Two teal neighbors inside the circle.

Count: 1 red (Class B) One coral neighbor inside the circle.

Majority vote: 2 vs 1 Result: classified as Class A!

Distance Metrics — How We Measure 'Closeness'

KNN's entire logic depends on distance. The most common metrics:

Euclidean Distance

The 'straight line' distance

Like measuring with a ruler on a map — as the crow flies.

```
d = sqrt( (x2-x1)^2 + (y2-y1)^2 )
```

Best for: continuous numerical data (height, weight, age)

`A=(1,2)  B=(4,6) → d = sqrt(9+16) = 5`

Manhattan Distance

The 'city block' distance

Like a taxi in NYC — can only go up/down or left/right.

```
d = |x2-x1| + |y2-y1|
```

Best for: grid-like data (pixels, text features, chess)

`A=(1,2)  B=(4,6) → d = 3 + 4 = 7`

Minkowski Distance

A generalized form (p controls which type)

A dial that shifts between Manhattan and Euclidean distance.

```
d = ( |x2-x1|^p + |y2-y1|^p
)^(1/p)
```

p=1 → Manhattan | p=2 → Euclidean | tune p freely

`Used when you want to tune the distance sensitivity`

Important: Always NORMALIZE features before using KNN — otherwise large-scale features dominate all distance calculations!

Choosing K — The Most Important Decision

K is the number of nearest neighbors used to vote. It is a hyperparameter — chosen before training. Wrong K = poor model.

K = 1 (Too Low)

OVERFITTING

Model memorises every training point.

Very sensitive to noise.

Works on training data but fails on new examples. Jagged, irregular boundaries.

K = 5 to 7 (Good)

BALANCED (usually best)

Smooth decision boundary.

Good generalization to new data.

The sweet spot for many real datasets — always verify with cross-validation!

K = 100 (Too High)

UNDERFITTING

Too many neighbors = biased toward majority class.

Decision boundary becomes overly smooth and flat.

Tip: Use ODD values of K for binary classification to avoid ties (K=3,5,7). Use Cross-Validation to find the best K.

Feature Scaling — Why It's Critical for KNN

The Problem: KNN uses distance. If Feature A ranges 0–1 and Feature B ranges 0–100,000 then Feature B completely dominates ALL distance calculations, making Feature A irrelevant!

Concrete Example: Predicting if someone buys a product

Feature	Raw Value	After Scaling (0–1)	Why scale?
Age	25 years	0.25	Original range was 0–100
Income	$75,000	0.75	Original range was $0–$100,000
Past Purchases	3 items	0.30	Original range was 0–10

Two Common Scaling Methods:

Min-Max Normalization → scales to [0,1]
formula: (x - min) / (max - min)

StandardScaler (Z-score) → mean=0, std=1
formula: (x - mean) / std_deviation

KNN in Python — scikit-learn Implementation

```python
# 1. Import libraries
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

# 2. Split into train/test sets
X_train, X_test, y_train, y_test = \
  train_test_split(X, y, test_size=0.2)

# 3. Scale features (CRITICAL!)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test  = scaler.transform(X_test)

# 4. Create & train KNN model
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)

# 5. Evaluate on test data
accuracy = knn.score(X_test, y_test)
print(f'Accuracy: {accuracy:.2f}')
```

Code Breakdown:

Imports

sklearn gives us ready-made KNN tools — no need to code distance from scratch.

Train/Test Split

We hold back 20% of data to evaluate how well the model generalizes to NEW, unseen examples.

Scaling (Critical!)

fit_transform on train only. Use .transform() (not fit_transform) on test — avoid data leakage!

n_neighbors=5

.fit() stores training data — KNN is a lazy learner, zero computation at training time.

Evaluate

.score() returns accuracy 0.0–1.0 on test set. Compare to baseline to judge real performance.

'Lazy Learner': KNN does ZERO computation during .fit() — it just stores the data. All work happens at prediction time.

Finding the Best K with Cross-Validation

We cannot just guess K — we systematically test multiple values and pick the one with best performance:

```
from sklearn.model_selection import cross_val_score

k_values = range(1, 21)    # test K from 1 to 20
accuracies = []

for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k)
    scores = cross_val_score(knn, X, y, cv=5)
    accuracies.append(scores.mean())

best_k = k_values[accuracies.index(max(accuracies))]
```

Accuracy vs. K value:

What is 5-Fold Cross-Validation?

Split training data into 5 equal parts. Each fold takes a turn as the test set (5 rounds total). Average the 5 accuracy scores.

Fold 1 (Train)	**Fold 2 (TEST)**	Fold 3 (Train)	Fold 4 (Train)	Fold 5 (Train)

KNN for Regression — Not Just Classification!

KNN can predict numerical values too — instead of majority vote, it takes the AVERAGE of neighbor values.

	KNN Classification	KNN Regression
Goal	Predict a CATEGORY	Predict a NUMBER
Output	e.g. Spam / Not Spam	e.g. $350,000 house price
Prediction method	MAJORITY VOTE of K neighbors	AVERAGE of K neighbor values
sklearn class	KNeighborsClassifier	KNeighborsRegressor
Performance metric	Accuracy, F1-score, ROC-AUC	RMSE, MAE, R-squared

KNN Regression — Python Code:

```
from sklearn.neighbors import KNeighborsRegressor
knn_reg = KNeighborsRegressor(n_neighbors=5)
knn_reg.fit(X_train, y_train)    predictions = knn_reg.predict(X_test)
```

Example: Predict house price based on 5 nearest houses with known prices → average their prices = prediction.

Real-World Applications of KNN

Recommendation Systems

Netflix / Spotify find users with similar tastes and recommend what similar users watched or listened to.

Medical Diagnosis

Predict disease from patient symptoms. Patients with similar profiles likely have the same condition.

Spam Detection

Classify email as spam / not spam. New email compared against thousands of labeled examples.

Fraud Detection

If a transaction looks very different from a user's usual patterns — it may be fraud.

Stock Prediction

Find historical periods with similar patterns to current market data to predict next movement.

Image Recognition

Compare pixel features of new image against labeled training images to classify objects.

KNN is especially popular in recommendation systems — the concept of 'similarity between users' maps directly to nearest-neighbor search.

KNN — Advantages & Disadvantages

ADVANTAGES

Simple & Intuitive

Very easy to understand and explain to non-technical stakeholders.

No Training Phase

KNN is a lazy learner — just stores data. Training time is essentially zero.

Naturally Multiclass

Handles 3+ classes with no modifications needed whatsoever.

Non-parametric

No assumptions about data distribution. Handles non-linear decision boundaries.

Easy to update

Add new training examples without any retraining — just add to the dataset.

DISADVANTAGES

Slow at prediction time

Must calculate distance to ALL training points for each new query. Scales as O(n).

High memory usage

Stores entire training dataset in memory — not scalable to millions of records.

Sensitive to scale

Features must be normalized or large-scale features dominate distances.

Sensitive to noise

Outliers and irrelevant features distort distance calculations badly.

Curse of dimensionality

Performance degrades sharply with many features — distances lose meaning.

Decision Guide — When Should You Use KNN?

USE KNN when:

- Dataset is SMALL (< 10,000 rows)
- Features are meaningful and few (< 20)
- You want a simple, interpretable baseline
- Data is well-labeled and cleaned
- Non-linear decision boundaries exist

AVOID KNN when:

- Dataset has millions of rows
- You have many features (100+)
- Real-time prediction speed required
- Data has many outliers or noise
- Memory on your system is limited

Rule of Thumb: Use KNN as a **baseline model** first. It is fast to set up and gives you a reference score. If it performs well — great! If not, upgrade to Random Forest, SVM, or Neural Networks which scale better.

Session Summary — Key Takeaways

01

KNN classifies by similarity

Find K nearest neighbors in feature space → majority vote (classification) or average (regression).

02

Distance is the engine

Euclidean distance is most common. Always normalize/scale features — otherwise distances are meaningless.

03

K controls complexity

Small K → overfit (too complex). Large K → underfit (too simple). Use cross-validation to tune.

04

KNN is a 'lazy learner'

Zero computation at training. All work at prediction time → fast to start, slow to predict at scale.

05

Scales poorly with size

For very large data or many features, prefer Random Forest, SVM, or deep learning.

Great work!

You now understand KNN.

Coming up in Session 06:

- Decision Trees — if/else logic visualized as a tree
- Random Forests — ensemble of many decision trees
- Model Evaluation — accuracy, precision, recall, F1

AI MASTER CLASS